Simple Local Search Improves 33 Published Upper Bounds for Balanced Latin Rectangles

Zhengyu Li
Georgia Institute of Technology
brian.li@gatech.edu

Vijay Ganesh
Georgia Institute of Technology
vganesh45@gatech.edu

Abstract

A $k \times n$ Latin rectangle is balanced when every pair of symbols has the same total distance, summed over the rows. We evaluate simulated annealing with an $O(n)$ within-row swap and an adaptive penalty for column conflicts. Against Table 1 of the CP 2025 study, it improves 33 of 51 non-square instances with $8 \leq n \leq 13$ and matches the other 18. The reduction averages 18.5% over the improved instances and reaches 39.4% (12×13: $274.0 \to 166.0$). These final values use up to 16 allocated CPU minutes per instance across separate runs. In a separate first-improvement experiment on the 33 improved instances, all 165 runs beat the reference bound: the pooled median is 0.12 CPU seconds and the maximum is 143 seconds. Four older SAT targets are reached or improved within 0.07 seconds in all 5 runs per target. These historical timing measurements are not controlled speedup comparisons: hardware differs, and CP 2025 reports budgets rather than time to the reference quality. We provide the rectangles, an independent exact checker, and code. The improved values are feasible upper bounds, not optimality certificates.

1 Introduction

Latin squares and rectangles are classical tools of experimental design. In a field trial, spatial autocorrelation can bias a Latin layout when some pairs of treatments are always planted closer together than others [van Es and van Es, 1993]. *Spatially balanced* Latin squares address this source of spatial imbalance, because every pair of symbols has the same total distance [Gomes and Sellmann, 2004, Le Bras et al., 2012]. Díaz et al. [2017] extended the idea to $k \times n$ rectangles. Balanced rectangles exist only in restricted cases. Díaz et al. therefore defined the *imbalance* of a rectangle and asked for rectangles of minimum imbalance. They compared CSP, local search and MIP, and reported that local search outperformed MIP on larger instances with $k > 3$. Their local search swaps whole columns and samples row subsets of cyclic squares. Our within-row moves explore a different space and may cross column-infeasible states; we do not claim to introduce local search for this problem.

Two recent studies attacked this benchmark. Peruvemba Ramaswamy and Szeider [2023] used SAT encodings, which also certify optimality with DRAT proofs. Voboril et al. [2025] used streamliners generated by large language models (LLMs) and evaluated them with Gurobi. This approach improved the best known values on most open instances. Balanced Latin rectangles also serve as a test problem for LLM agents that reformulate constraint models [Voboril and Szeider, 2026].

Contribution. We revisit the problem with a deliberately simple method. It is simulated annealing [Kirkpatrick et al., 1983] over rows that are permutations, with a swap move that costs

$O(n)$ time to evaluate. A penalty for repeated symbols in a column adapts its weight during the search, in the spirit of strategic oscillation [Glover and Hao, 2011]. None of these ingredients is new. Our contribution is a set of independently verified improved upper bounds. Throughout, *record* means the reference value in Table 1 of Voboril et al. [2025]; it does not assert priority over every later or unpublished result. The search improves 33 of these 51 values and matches the rest. The timing experiment concerns first improvement over the reference, not the time to obtain the final values. These results motivate including a strong problem-specific local-search baseline alongside exact and LLM-guided methods.

2 Problem

A $k \times n$ Latin rectangle ($k \le n$) is a $k \times n$ array over the symbols $\{1, \dots, n\}$ in which no symbol repeats in a row or in a column. Write $c_i(u)$ for the column of symbol u in row i. Following Peruvemba Ramaswamy and Szeider [2023], the distance of two symbols and the imbalance of L are

$$d(u,v) = \sum_{i=1}^{k} |c_i(u) - c_i(v)|, \qquad I(L) = \sum_{1 \le u < v \le n} \left| d(u,v) - \tfrac{k(n+1)}{3} \right|.$$

The target $k(n+1)/3$ is the average of $d(u,v)$ over all pairs, because every row contributes $\sum_{u<v} |c_i(u) - c_i(v)| = (n+1)n(n-1)/6$. The value $3I(L)$ is always an integer. As in the literature, we write imbalances as $x.0$, $x.3$ and $x.6$ for x, $x + \frac{1}{3}$ and $x + \frac{2}{3}$. The goal is a rectangle of minimum imbalance for given k and n.

3 Method

Search space and move. The state is k permutations of the symbols, one per row. We fix the first row to the identity. This loses no generality, because a relabelling of the symbols does not change the imbalance. A move picks a row $i \ge 2$ and two columns $j_1 \ne j_2$, and it swaps the symbols $a = L[i][j_1]$ and $b = L[i][j_2]$. Rows stay permutations, but the swap can create a repeated symbol in a column.

Incremental evaluation. We keep the matrix D of all pair distances. A swap in row i moves only a and b. So only the $2(n-2)$ distances $d(a,x)$ and $d(b,x)$ with $x \notin \{a,b\}$ change, and $d(a,b)$ stays the same. For each such x, with $p_x = c_i(x)$,

$$d'(a,x) = d(a,x) - |j_1 - p_x| + |j_2 - p_x|, \qquad d'(b,x) = d(b,x) - |j_2 - p_x| + |j_1 - p_x|.$$

The search therefore computes the exact change of $3I(L)$ in $O(n)$ integer operations. A recomputation from scratch costs $O(kn^2)$.

Column conflicts. Let V be the number of excess symbol occurrences over all columns. Per-column counters give V in $O(1)$ time per move. The search minimizes $3I(L) + W \cdot V$, and it records only rectangles with $V = 0$. The weight W starts at k. Every 2^{16} proposals, including proposals with identical columns, the search multiplies W by 1.02 if the current state has conflicts and by 0.99 otherwise, clamping it to $[1, 4n^2]$. This allows exploration through infeasible states while retaining only feasible incumbents. We have not isolated the benefit of this adaptation in a controlled ablation.

Annealing schedule. The search starts from the cyclic rectangle $L[i][j] = i + j \bmod n$. It accepts a move with change Δ if $\Delta \leq 0$, and otherwise with probability $e^{-\Delta/T}$. The temperature starts at $T_0 = 2k$ and is multiplied by 0.9999995 after each nontrivial proposal, accepted or rejected, with a floor of 0.05. The implementation samples columns independently and skips proposals with $j_1 = j_2$. At checks every 2^{16} proposals, it reheats after more than $4 \cdot 10^7$ proposals since an improvement or the last reheat. Reheating also resets this counter. Each run is single-threaded. Its CPU timer starts after initialization and excludes process launch, output and external verification. Budget checks can overshoot by a check interval. Before reporting the best rectangle, the compact C implementation recomputes its imbalance from scratch.

4 Experiments

Setup. We use the instances in Table 1 of Voboril et al. [2025], with $8 \leq n \leq 13$ and $2 \leq k \leq n-1$. This gives 51 instances. We omit squares ($k = n$), because different structure governs that case [Xia et al., 2026]. The record of each instance is the best value reported there. It already includes earlier results and four-hour Gurobi runs. We used an Apple M3 laptop (8 cores, 16 GB) and ran eight single-threaded runs at a time. Each instance got one run with a budget of 1 minute and one run with 5 minutes, with different seeds. The larger instances ($k \geq 4$, $n \geq 9$) also got one 10-minute run. We report the best rectangle per instance, so the largest allocated total is 16 CPU minutes per instance. The 137 runs have a total allocated budget of 10 hours 56 minutes; these are CPU budgets, not elapsed wall times. The timing CSV preserves the original measurements. Their per-run stdout, stderr and build provenance were not archived, so they cannot be reconstructed from that CSV alone; the revised timing runner archives these for future experiments.

Verification. An independent Python program checks every reported rectangle. We wrote it from the definition above. It uses exact rational arithmetic and shares no code with the solver. As a finite cross-check of the published quantity, we ran it on all fourteen rectangles printed in [Peruvemba Ramaswamy and Szeider, 2023, Voboril et al., 2025]. These include the eight CP 2025 records of their Figure 6, up to 12×13. The checker reproduces every reported imbalance exactly.

Results. Table 1 shows the outcome. The search improves 33 instances, matches 18, and is worse on 0. All improved instances have $n \geq 9$ and $k \geq 3$. The search matches the record on all instances with $n = 8$ or $k = 2$, on all instances with $k = 3$ except 3×12, and on 4×10, 5×10 and 6×12. Several of these records are proven optimal [Peruvemba Ramaswamy and Szeider, 2023]. The maximum reduction is 39.4% for 12×13; the mean over the 33 improved instances is 18.5%. The one-minute runs alone improve 33 instances. Over the 35 instances with all three budgets, mean reductions are 13.3%, 15.8% and 16.8% for the separate one-, five- and ten-minute runs. Because each budget uses a different seed, these are not checkpoints of one trajectory or an isolated estimate of the effect of time. Individual outcomes need not improve monotonically with budget.

Speed. To measure speed directly, we ran the search on every improved instance with 5 further seeds. Each run stopped as soon as its imbalance fell below the record, with a cap of five CPU minutes (Figure 1b). All 165 runs beat the record. The pooled median across all 165 runs is 0.12 seconds, and the slowest run took 143 seconds. The median per instance is below one second on 23 of the 33 instances. It is below one minute on all of them, with a maximum of 54 seconds for 9×10. CP 2025 allocates 4 to 9 hours per approach after a four-hour Gurobi baseline. Those budgets are

$n\backslash k$	2	3	4	5	6	7	8	9	10	11	12
8	40.0	36.0	32.0	30.0	24.0	28.0					
9	65.3	56.0	**53.3** ~~56.0~~	**48.6** ~~52.6~~	**40.0** ~~48.0~~	**43.3** ~~51.3~~	**42.6** ~~53.3~~				
10	92.0	80.0	78.6	66.6	**60.0** ~~70.0~~	**58.0** ~~72.0~~	**58.6** ~~82.6~~	**64.0** ~~70.0~~			
11	124.0	114.0	**104.0** ~~112.0~~	**94.0** ~~116.0~~	**88.0** ~~108.0~~	**84.0** ~~118.0~~	**78.0** ~~114.0~~	**84.0** ~~120.0~~	**100.0** ~~110.0~~		
12	168.0	**152.0** ~~154.0~~	**149.3** ~~154.6~~	**140.0** ~~145.3~~	120.0	**121.3** ~~164.0~~	**116.6** ~~148.0~~	**106.0** ~~154.0~~	**124.0** ~~174.6~~	**136.6** ~~147.3~~	
13	218.6	198.0	**201.3** ~~211.3~~	**191.3** ~~210.6~~	**166.0** ~~204.0~~	**164.0** ~~203.3~~	**160.0** ~~220.6~~	**152.0** ~~214.0~~	**154.6** ~~240.6~~	**168.0** ~~224.0~~	**166.0** ~~274.0~~

Table 1. Reference and achieved imbalance of $k \times n$ Latin rectangles. Bold: new value from this work, with the record [Voboril et al., 2025] struck through below it. Plain: record that our search matches.

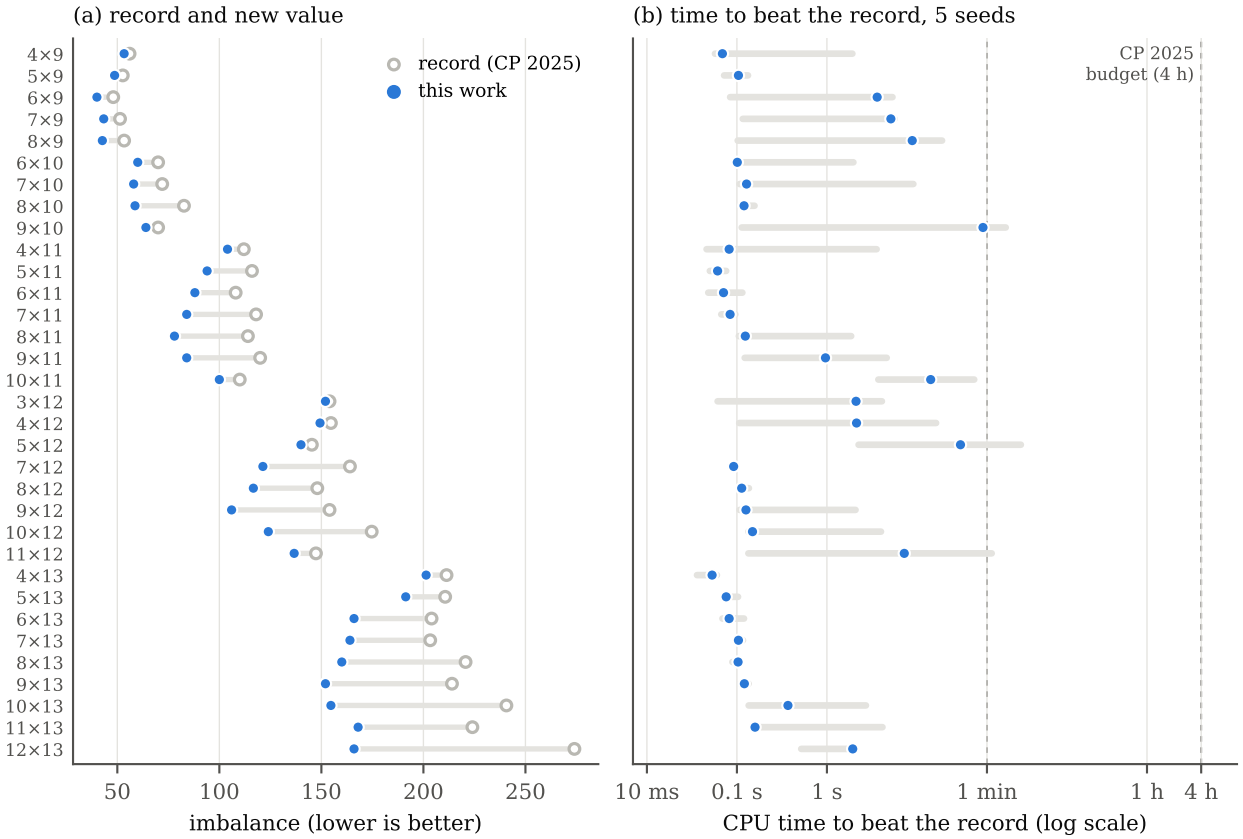


Figure 1. All 33 improved instances. (a) Record (open circle) and new imbalance (filled). (b) CPU time until the search beats the record, on one core: median (dot) and range (bar) over 5 seeds, log scale. The dashed lines mark one minute and a CP 2025 four-hour budget, not its time to the reference bound.

context, not measurements of time to the same quality. The timing experiment does not measure time to the final bounds in Table 1.

Peruvemba Ramaswamy and Szeider [2023] report the SAT solving time for four rectangles that set records at that time. The times range from 32 minutes (3×10, imbalance 84) to 5.5 hours (6×11, imbalance 124). Figure 2 shows all four. In every run, our search reaches or improves the target imbalance within 0.07 seconds. This is a cross-study comparison of reported times, not a calibrated speedup.

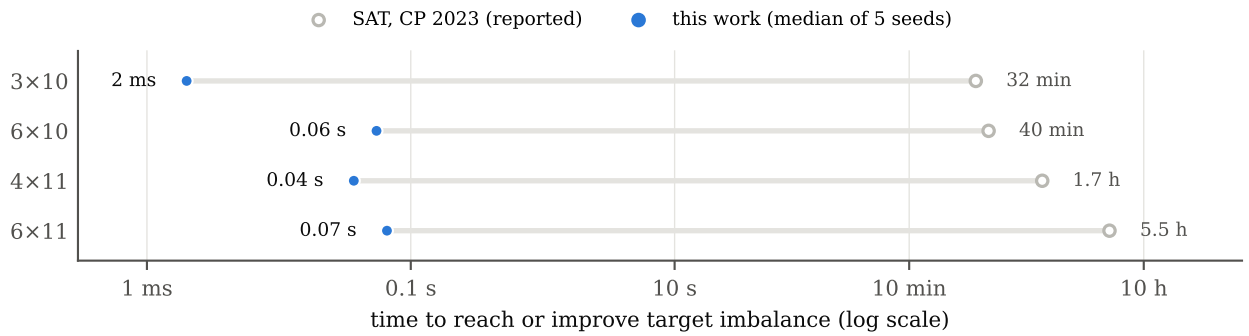


Figure 2. Time to reach or improve the target imbalance that the SAT approach of Peruvemba Ramaswamy and Szeider [2023] reports, next to its reported solving time. Ours: median of 5 seeds. Log scale.

5 Discussion

Interpretation. Incremental evaluation makes each proposal inexpensive, while penalized column conflicts allow transitions between feasible rectangles. Weak LP relaxations and restrictions imposed by streamliners are possible explanations for the relative results, but we did not measure relaxation gaps or isolate these mechanisms. The experiments establish achievable bounds, not a causal explanation of performance.

Relation to earlier local search. Local search for balanced Latin designs predates this work [Smith et al., 2005, Díaz et al., 2017]. In particular, the 2017 rectangle study already found local search competitive with, and on larger cases better than, MIP. Its whole-column swaps and selection of rows from cyclic squares differ from our within-row neighborhood and infeasible intermediate states. We have not rerun that method head-to-head, so we cannot attribute our gains to one of these differences. The newer bounds provide a useful reference for exact, LLM-guided and local-search approaches; they do not establish a general ranking of these algorithm families.

Timing comparability. The original measurements used an Apple M3 laptop; the earlier studies used Intel Xeon E5-2640 v4 servers. We did not calibrate their relative performance or rerun the earlier pipelines. CP 2025 does not report time to each reference bound. CP 2023 reports finding times for satisfiable decision instances, but its solver also supports infeasibility proofs, which our heuristic cannot provide. Initialization, launch and verification are outside our reported search timer. Consequently neither comparison establishes a hardware-controlled speedup.

Limitations. The new values are upper bounds. We prove nothing about optimality, and the size of the improvements suggests that the remaining gap to the optimum may still be large. The final records come from one run per instance and budget. Only the speed experiment uses several seeds. We also lack controlled penalty and neighborhood ablations and a direct comparison with the earlier local-search implementation. Finally, the benchmark stops at $n = 13$, and larger instances are an obvious next step.

6 Conclusion

A short, standard simulated-annealing search improves 33 of the 51 reference imbalance values for balanced Latin rectangles in CP 2025 Table 1, and it matches the rest. All 165 first-improvement

runs on the 33 improved instances beat their reference within 143 CPU seconds in the historical timing data. This is distinct from the final-quality experiment and does not establish a controlled speedup. Natural next steps follow from this.

- Prove lower bounds or optimality for the improved instances, for example with the SAT approach of Peruvemba Ramaswamy and Szeider [2023].
- Extend the benchmark to larger instances.
- Compare exact, LLM-guided and local-search methods on equal hardware.

Data and code. The artifacts accompanying this manuscript include rectangles, checker, solver and scripts: `https://github.com/BrianLi009/Balanced-Latin-Rectangles`.

References

Mateo Díaz, Ronan Le Bras, and Carla P. Gomes. In search of balance: The challenge of generating balanced Latin rectangles. In *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2017)*, volume 10335 of *Lecture Notes in Computer Science*, pages 68–76. Springer, 2017. doi: 10.1007/978-3-319-59776-8_6.

Fred Glover and Jin-Kao Hao. The case for strategic oscillation. *Annals of Operations Research*, 183(1):163–173, 2011. doi: 10.1007/s10479-009-0597-1.

Carla P. Gomes and Meinolf Sellmann. Streamlined constraint reasoning. In *Principles and Practice of Constraint Programming (CP 2004)*, volume 3258 of *Lecture Notes in Computer Science*, pages 274–289. Springer, 2004. doi: 10.1007/978-3-540-30201-8_22.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

Ronan Le Bras, Carla P. Gomes, and Bart Selman. From streamlined combinatorial search to efficient constructive procedures. In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, pages 499–506, 2012. doi: 10.1609/aaai.v26i1.8147.

Vaidyanathan Peruvemba Ramaswamy and Stefan Szeider. Proven optimally-balanced Latin rectangles with SAT (short paper). In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, volume 280 of *LIPIcs*, pages 48:1–48:10. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi: 10.4230/LIPIcs.CP.2023.48.

Casey Smith, Carla P. Gomes, and Cèsar Fernández. Streamlining local search for spatially balanced Latin squares. In *Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence (IJCAI-05)*, pages 1539–1540, 2005.

H. M. van Es and C. L. van Es. Spatial nature of randomization and its effect on the outcome of field experiments. *Agronomy Journal*, 85(2):420–428, 1993. doi: 10.2134/agronj1993.00021962008500020046x.

Florentina Voboril and Stefan Szeider. Improving constraint models with LLM agents, 2026. arXiv:2608.08127.

Florentina Voboril, Vaidyanathan Peruvemba Ramaswamy, and Stefan Szeider. Balancing Latin rectangles with LLM-generated streamliners. In *31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*, volume 340 of *LIPIcs*, pages 36:1–36:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. doi: 10.4230/LIPIcs.CP.2025.36.

Hai Xia, Carla P. Gomes, Bart Selman, and Stefan Szeider. Agentic neurosymbolic collaboration for mathematical discovery: A case study in combinatorial design, 2026. arXiv:2603.08322.